# TEQ: Trainable Equivalent Transformation for Quantization of LLMs

**Wenhua Cheng** and **Yiyang Cai** and **Kaokao Lv** and **Haihao Shen**
Intel
{wenhua.cheng, yiyang.cai, kaokao.lv, haihao.shen}@intel.com

arXiv:2310.10944v1 [cs.CL] 17 Oct 2023

## Abstract

As large language models (LLMs) become more prevalent, there is a growing need for new and improved quantization methods that can meet the computationalast layer demands of these modern architectures while maintaining the accuracy. In this paper, we present TEQ, a trainable equivalent transformation that preserves the FP32 precision of the model output while taking advantage of low-precision quantization, especially 3 and 4 bits weight-only quantization. The training process is lightweight, requiring only 1K steps and less than 1‰ of the original model's trainable parameters. Furthermore, the transformation does not add any computational overhead during inference. Our results are on-par with the state-of-the-art (SOTA) methods on typical LLMs. Our approach can be combined with other methods to achieve even better performance. The code is available at https://github.com/intel/neural-compressor.

## 1 Introduction

Large language models (LLMs) have not only shown breakthrough performance in a wide range of benchmarks and tasks but played an increasingly important role in daily life, e.g., ChatGPT (OpenAI) in information retrieval and Copilot (Github) in programming. However, as LLMs' model size keeps growing dramatically, their significant memory footprint and heavy computation requirements become a major bottleneck of their usage.

One of the most promising ways to alleviate this challenge is quantization, which can reduce storage and computational overhead. Quantization converts high-bit floating-point data to lower-bit representations, and it has become an effective model compression technique.

Quantization methods can generally be divided into two categories: quantization aware training (QAT) (Shen et al., 2021; Zhuang et al., 2021; Gong et al., 2019; Esser et al., 2019; Louizos et al., 2018) and post-training quantization (PTQ) (Frantar et al., 2022; Li et al., 2022; Xiao et al., 2022; Wei et al., 2022; Frantar and Alistarh, 2022; Hubara et al., 2021; Nagel et al., 2020; Hassibi et al., 1993; LeCun et al., 1989). Their effectiveness has been validated for a wide range of models. However, several issues still need to be addressed, especially for LLMs. QAT simulates the quantization behavior in the training/finetuning phase, but such a process is very costly for LLMs due to their unprecedented parameter scale. In contrast, PTQ requires no training and thus has drawn rising attention. However, PTQ is prone to large accuracy drops, especially for extreme low-bit quantization. This provides LLMs' PTQ methods with great opportunities for improvement.

Lower-bit quantization (e.g., Int4, W4) has recently been widely discussed since memory bandwidth is becoming the main bottleneck of LLMs. However, most existing works focus on computer vision models (He et al., 2016; Howard et al., 2017) that are much smaller than current popular LLMs such as BLOOM-176B(Scao et al., 2022), OPT-175B(Zhang et al., 2022). Other extreme quantization methods (Bai et al., 2020; Zhang et al., 2020) rely on the knowledge distillation technique, introducing extra overhead. GPTQ(Frantar et al., 2022) tunes the weights based on optimal brain surgeon(Hassibi et al., 1993) and successfully achieves low-bit quantization on LLMs with low computation overhead.

Our proposed method reduces the compression error by introducing a trainable equivalent transformation (Fig. 1), which keeps the mathematical equivalency of model output at FP32 precision. Moreover, the training cost is significantly low, only 1k steps of batch size 1 with around less than one-thousandth trainable parameters of the original models. Also, our method is orthogonal to current popular LLMs quantization methods, and better accuracy results could be achieved by combining

ours with them.

In summary, the contribution of this paper is threefold:

- We introduce a trainable equivalent transformation for the quantization of LLMs, which keeps the model output unchanged at FP32 precision. Besides, the training is quite lightweight.
- Experimental results show our method could achieve results on par with or better than the SOTA methods.
- We also show that our method could be combined to get the new SOTA performance.

In the following, we first briefly introduce the work related to ours in Section 2. We then present the trainable equivalent transformation in Section 3. Experiments and conclusion are described in Sections 4 and 5 respectively.

## 2 Related Work

**Quantization-aware Training.** QAT methods are widely used in model compression. By enabling finetuning process, quantized models' accuracy can often be on par with or even better than those of original models. (Louizos et al., 2018) introduce a differentiable quantization procedure by converting original weights and activations' distribution to categorical distributions. OQAT (Shen et al., 2021) proposes a combined training scheme of architecture and quantization to acquire many quantized models. Afterward, they are converted to lower-bit models and optimized. (Zhuang et al., 2021) propose a progressive quantization scheme by quantizing activations after weights. Indeed, QAT methods are popular in relatively small-scale models, but their application in LLMs is limited due to the expensive training or even fine-tuning costs as mentioned in Section 1.

**Post-training Quantization.** A large number of post-training methods quantize weights step by step and modify unquantized weights to compensate for errors produced by previously quantized weights. Optimal Brain Damage (OBD) (LeCun et al., 1989) uses second-derivative information (Hessian-based estimation) to predict the effect of weights' perturbation analytically. Optimal Brain Surgeon (OBS) (Hassibi et al., 1993) applies such an idea by devising a second-order framework for weight pruning. Afterward, Optimal Brain Quantization (OBQ) migrate OBS's pruning framework to quantization since pruning and quantization share the common idea of introducing perturbation in original models. Finally, GPTQ (Frantar et al., 2022) improves the original framework's efficiency by fixing the quantization order within the layer and calculating the Hessian matrix's Cholesky decomposition before quantization. Other PTQ methods use a better rounding scheme than commonly used rounding-to-nearest (RTN). AdaRound (Nagel et al., 2020) learns a rounding scheme using mean squared error (MSE) for layer-wise activation. AQuant (Li et al., 2022) adds a learnable border function for activation quantization.

**Large Language Model Quantization.** Researchers are devoting efforts to compression methods particularly designed for LLMs as more open-source releases are available. LLM.int8() (Dettmers et al., 2022) discovers peak values in activation outliers' particular channels. It proposes methods to ensure that these channels are kept in higher precision. SmoothQuant (Xiao et al., 2022) addresses the issues mentioned above by migrating difficulties from activation to weights with a handcrafted equivalent transformation. ZeroQuant (Yao et al., 2022) devises an end-to-end quantization and inference pipeline with a novel layer-wise knowledge distillation algorithm. However, the largest model it has quantized has only 1.3B parameters. GPTQ (Frantar et al., 2022) tunes the weights based on optimal brain surgeon (Hassibi et al., 1993) and successfully achieves low-bit quantization on LLMs with low computation overhead. More recent, AWQ (Lin et al., 2023) propose to search the optimal scales to protect parts of weights, since they can significantly reduce the error caused by quantization.

## 3 Methodology

Figure 1 presents a schematic illustration of equivalent transformation. In the following, we introduce the quantization process first. Consider a feed-forward neural network comprised of $L$ layers, which perform matmul or convolution operations. We only consider the matmul layer for simplicity since our method could be easily extended to convolution layers. The $l^{th}$ matmul operation can be denoted by $y_l = w_l \cdot x_l$. In which $w_l$ and $x_l$ are the weights and activation(input), and $y_l$ is the corresponding output. To quantize a tensor, a

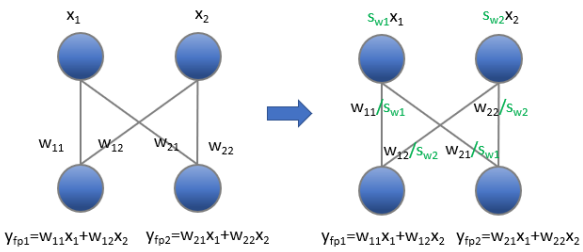



Figure 1: A schematic illustration of TEQ, where $s_{w1}$ and $s_{w2}$ are trainable parameters. A per-channel scale is multiplied at activations while an inverse scale is multiplied at weights, which could keep the output equivalent.

quantization op presented below could be applied.

$$Q(v) = clip(\left[\frac{v}{s}\right], -n, n), n \in \mathbb{N} \tag{1}$$

where $s$ denotes the quantization scale parameter and $[\cdot]$ denotes the round-to-nearest (RTN) operation, while $-n$ and $n$ denote the integer thresholds for clipping. We ignore the zero point for simplicity. For a normal int8 quantization, i.e., W8A8, we need to quantize activation and weights both. And for weight-only quantization, only the weights need to be quantized. Finally, a de-quantization operation will be appended to reconstruct the float output, normally not equal to $y_l$. In summary, the $L_l$'s output after normal quantization is converted to:

$$\hat{y}_l = Q^{-1}(Q(w_l) \cdot Q(x_l)) \tag{2}$$

where $\hat{y}_l$ denotes the $L_l$'s reconstructed output after quantization. The value of $(y_l - \hat{y}_l)^2$ is usually named as quantization loss.

### 3.1 Trainable Equivalent Transformation

PTQ tends to cause a noticeable accuracy drop as mentioned before. SmoothQuant (Xiao et al., 2022) and AWQ (Lin et al., 2023) rely on hand-crafted rules to migrating quantization difficulties of weights and activations. However, these rules often fall into sub-optimal solutions, which cannot minimize error caused by quantization. To alleviate this issue, we introduce a trainable equivalent transformation that enforces the Fp32 output as the same but greatly improves the quantization robustness. To be more specific, suppose the shape of $w_l$ is $c_l^{in} \times c_l^{out}$, which stands for their respective input and output channel numbers. For each layer $L_l$, we can multiply a per-channel scaling vector $s_l \in \mathbb{R}^{c_l^{in}}$ for weights and append a corresponding inverse scale vector for activation. Mathematically, this can be restated as

$$y_l = w_l \cdot diag(s_l) \cdot diag(s_l)^{-1} \cdot x_l \tag{3}$$

operator $diag(\cdot)$ denotes converting a column/row vector to a diagonal matrix whose eigenvalues are identical to the original vector's elements.

$$diag\left(\begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_n \end{bmatrix}\right) = \begin{bmatrix} s_1 & & & \\ & s_2 & & \\ & & \ddots & \\ & & & s_n \end{bmatrix} \tag{4}$$

Our observation shows the optimal $s_w$ is useful to reduce the quantization loss. Therefore, we quantize the transformed model rather than the original one.

The transformation has two per-channel scale operations, which will introduce computation overhead. We fuse the weight scale to the weight itself. For the activation scale, following (Xiao et al., 2022), we fuse it to the previous layers, such as layernorm(Ba et al., 2016), batchnorm(Ioffe and Szegedy, 2015) and etc. In all our experiments, we only apply the transformation to the layer whose scales could be fused, which introduces no extra overhead at deployment.

### 3.2 Training Details

We train the scales $s_l$ because there is little knowledge of the best equivalent transformation due to various models and quantization configurations. It's worth mentioning that the count of trainable scales is much less than the model's parameters, and the model weights are frozen.

To train the transformation scales, we follow the basic QAT to simulate the quantization behavior, which could be denoted as

$$y_{l_q} = (Q^{-1}Q(w_l))(Q^{-1}Q(x_l)) \tag{5}$$

For weight-only quantization, activation quantization will be ignored. We adopt straight-through estimator (STE) (Bengio et al., 2013) to backward the gradients.

We use Adam(Kingma and Ba, 2014) optimizer, betas [0.9, 0.9], and weight decay 0. The learning rate is 1e-3 unless explicitly stated and the decay type is linear. We only train 1000 steps. We use the same loss function as the original one in the training phase. For example, CrossEntorpy loss is adopted for LLMs. The $s_l$ is usually initialized with 1. However, sometimes $1.0/sqrt(w_{cin})$ leads

| n_bits | Methods | OPT-6.7B | OPT-13B | BLOOM-3B | BLOOM-7B1 | LLAMA-7B | LLAMA-13B |
|---|---|---|---|---|---|---|---|
| 32 | FP32 | 64.97 | 65.54 | 55.65 | 60.29 | 68.87 | 71.06 |
| 4 | RTN | 62.99 | 64.17 | 53.17 | 57.80 | 67.41 | 68.86 |
| | GPTQ | 63.09 | 64.83 | **54.65** | 58.26 | 64.70 | **70.00** |
| | Ours | 63.30 | 64.91 | 53.83 | 58.93 | **67.71** | 69.55 |
| | Ours+GPTQ | **63.94** | **65.03** | 54.42 | **59.62** | 65.27 | 69.73 |
| 4_g128 | RTN | 64.04 | 64.88 | 54.91 | 59.32 | 67.87 | 70.88 |
| | GPTQ | 64.76 | **65.37** | **55.68** | 59.59 | 66.33 | 70.92 |
| | Ours | 64.11 | 64.87 | 54.98 | 59.35 | **68.10** | **71.00** |
| | Ours+GPTQ | **64.77** | 65.20 | 55.49 | **59.60** | 66.56 | 70.96 |

Table 1: The w4 average accuracy(↑) of four tasks, e.g., HellaSwag, WinoGrande, PIQA, and LAMBADA, in LM-eval. g denotes group size. "Ours+GPTQ" means we apply TEQ first and then apply GPTQ afterward. For LLAMA-7B, the result of GPTQ is w/o act-order. Results of act-order are shown in Appendix A.2.

| n_bits | Methods | OPT-6.7B | OPT-13B | BLOOM-3B | BLOOM-7B1 | LLAMA-7B | LLAMA-13B |
|---|---|---|---|---|---|---|---|
| 32 | FP32 | 10.86 | 10.12 | 13.48 | 11.36 | 5.68 | 5.09 |
| 4 | RTN | 12.10 | 11.32 | 14.75 | 12.09 | **6.29** | 5.53 |
| | GPTQ | 11.59 | **10.33** | 14.10 | **11.73** | 6.59 | **5.33** |
| | Ours | 11.68 | 10.59 | 14.72 | 12.21 | 6.30 | 5.50 |
| | Ours+GPTQ | **11.29** | 10.36 | **14.03** | 11.74 | 6.76 | 5.35 |
| 4_g128 | RTN | 11.16 | 10.32 | 13.85 | 11.60 | **5.97** | 5.26 |
| | GPTQ | **10.98** | **10.20** | **13.69** | **11.48** | 6.29 | **5.21** |
| | Ours | 11.11 | 10.28 | 13.82 | 11.58 | **5.97** | 5.26 |
| | Ours+GPTQ | 11.02 | 10.21 | **13.69** | **11.48** | 6.28 | **5.21** |

Table 2: The w4 perplexity(↓) on WikiText-2. For LLAMA-7B, the result of GPTQ is w/o act-order. Results of act-order are shown in Appendix A.2.

to better results, so we pick the better one in our experiments.

## 4 Experiments

In this section, we evaluate our proposed TEQ's in different aspects. Initially, we briefly introduce LLM architectures and tasks included in our evaluation. Secondly, we illustrate a detailed comparison of our method and other state-of-the-art (SOTA) methods, and both quantization accuracy and time are considered.

### 4.1 Experimental Settings

**Large Language Models.** We conduct our experiments on the most popular LLM architectures, including LLaMAs (Touvron et al., 2023), BLOOMs (Scao et al., 2022), and OPTs (Zhang et al., 2022). Parameter scalings ranging from million to billion are all included.

**Evaluation and Datasets.** We make assessments on several language tasks to satisfy the task-agnostic setting. Specifically, we report average accuracy result on four common sense reasoning tasks by leveraging lm-eval-harness(Gao et al., 2021), including HellaSwag (Zellers et al., 2019), WinoGrande (Sakaguchi et al., 2021), PIQA (Bisk et al., 2020) and LAMBADA (Paperno et al., 2016). Furthermore, we complement our evaluation with perplexity (PPL) analysis on WikiText2 (Merity et al., 2016), PTB (Marcus et al., 1994) as well as C4 (Raffel et al., 2020).

**Implementation Details.** Following GPTQ (Frantar et al., 2022), we focus on weight-only quantization and exclude the last layer When quantifying. We used a single HW accelerator to quantize models with a scale of around ten billion parameters. We use the same calibration dataset pile-10k[1] for a fair comparison.

**Baseline.** Our primary baseline is vanilla round-to-nearest quantization (RTN) which has a remarkable result at 4bits using a small group size of 128. We also compare with a state-of-the-art method GPTQ (Frantar et al., 2022).

### 4.2 Results

As mentioned above, we compare our results with RTN and the SOTA GTPQ(Frantar et al., 2022). Also, since our method is orthogonal to GPTQ, we report Ours+GPTQ as well, which applies TEQ first and then runs GPTQ official code[2] afterward. We mainly focus on the models around 10B which

[1]https://huggingface.co/datasets/NeelNanda/pile-10k
[2]https://github.com/IST-DASLab/gptq

| n_bits | Methods | OPT-6.7B | OPT-13B | BLOOM-3B | BLOOM-7B1 | LLAMA-7B | LLAMA-13B |
|---|---|---|---|---|---|---|---|
| 32 | FP32 | 64.97 | 65.54 | 55.65 | 60.29 | 68.87 | 71.06 |
| 3_g128 | RTN | 56.03 | 49.59 | 52.54 | 57.53 | 64.92 | 67.68 |
| | GPTQ | 62.98 | **64.68** | 53.41 | **58.12** | 58.29 | 68.73 |
| | Ours | 61.41 | 63.27 | 52.69 | 57.79 | **65.25** | 68.32 |
| | Ours+GPTQ | **63.16** | 64.60 | **53.71** | 58.00 | 59.27 | **69.15** |

Table 3: The 3 bits with group size 128 average accuracy(↑) of four tasks,e.g., HellaSwag, WinoGrande, PIQA, and LAMBADA, in LM-eval. g denotes group size. For LLAMA-7B, the result of GPTQ is w/o act-order. Results of act-order are shown in Appendix A.2.

| n_bits | Methods | OPT-6.7B | OPT-13B | BLOOM-3B | BLOOM-7B1 | LLAMA-7B | LLAMA-13B |
|---|---|---|---|---|---|---|---|
| 32 | FP32 | 10.86 | 10.12 | 13.48 | 11.36 | 5.68 | 5.09 |
| 3_g128 | RTN | 22.37 | 40.50 | 15.68 | 12.47 | 7.01 | 5.88 |
| | GPTQ | 11.42 | **10.51** | 14.67 | 11.99 | 8.28 | **5.64** |
| | Ours | 12.03 | 11.83 | 15.48 | 12.40 | **6.89** | 5.81 |
| | Ours+GPTQ | **11.40** | 10.52 | **14.64** | **11.98** | 7.71 | **5.64** |

Table 4: WikiText-2 perplexity(↓) of 3 bits with group size 128. For LLAMA-7B, the result of GPTQ is w/o act-order. Results of act-order are shown in Appendix A.2.

is commonly used.

**W4 Quantization.** We first evaluate TEQ on popular 4 bits quantization. Table 1 shows the lm-eval results of different LLM model architectures and parameter sizes. TEQ outperforms RTN in all cases except one. Comparing with GPTQ, TEQ shows better results in 6 out of 12 scenarios. After combining GPTQ, new state-of-the-art results could be achieved in 5 scenarios. In summary, TEQ could be helpful in 8 out of 12 scenarios. Table 8 shows the hyper-parameters that we used in the experiements.

We also evaluate WikiText2 ppl in table 2 w/o group size and group size 128. TEQ is better or on par with RTN. Similarly, the combined approach (Ours and GPTQ) shows comparable or better results than standalone GPTQ.

**W3 Quantization.** We also evaluate TEQ at weight with 3 bits. We only consider group size 128, because the performance drops a lot without group size and usually could not be deployed in practice. Similar to 4 bits evaluation, we report the lm-eval result and wikitext2 ppl result in table 3 and 4 respectively. TEQ outperforms RTN in all scenarios and is inferior to GPTQ on certain models. However, TEQ could bring improvement for 8 out of 12 scenarios if taking Ours+GPTQ into account.

**Quantization Time.** We report the quantization time in Table 5. We adopt Deepspeed[3] for 10B+ models due to the potential out-of-memory (OOM) issue. As TEQ needs training, our time cost is reasonably higher than GPTQ, especially when the model does not fit into the device memory. It's possible to reduce the time further by using more resources or optimizing the code, while it's out of scope.

| Models | GPTQ | Ours |
|---|---|---|
| OPT-6.7B | 841 | 1239 |
| OPT-13B | 1523 | 8737* |
| BLOOM-3B | 345 | 506 |
| BLOOM-7B1 | 661 | 1148 |
| LLAMA-7B | 712 | 1249 |
| LLAMA-13B | 1240 | 9501* |

Table 5: Quantization time in seconds for 4-bit weight quantization. * denotes DeepSpeed is adopted in training for 10B+ models.

**Analysis of Scales in TEQ.** We visualize the magnitude distribution histograms of $s_l$ for the layers to which TEQ can be applied. Figure 2 displays the results of models with $s_l$ initialized as scalar ones. Several conclusions can be drawn from these results. Most notably, the majority of trained scales remain close to their initial values (e.g., 1), typically within the range of [0.75, 1.25]. This suggests that even minor changes to the model can significantly reduce quantization loss. Additionally, some scales deviate considerably from 1, indicating the presence of "outlier" channels. Furthermore, scales in middle layers tend to remain closer to their initial values compared to other layers, suggesting that the first and last layers are more sensitive to the quantization loss. We also attach results of scales initialized with $1.0/sqrt(w_{cin})$ in Appendix A.5.

[3]https://github.com/microsoft/DeepSpeed

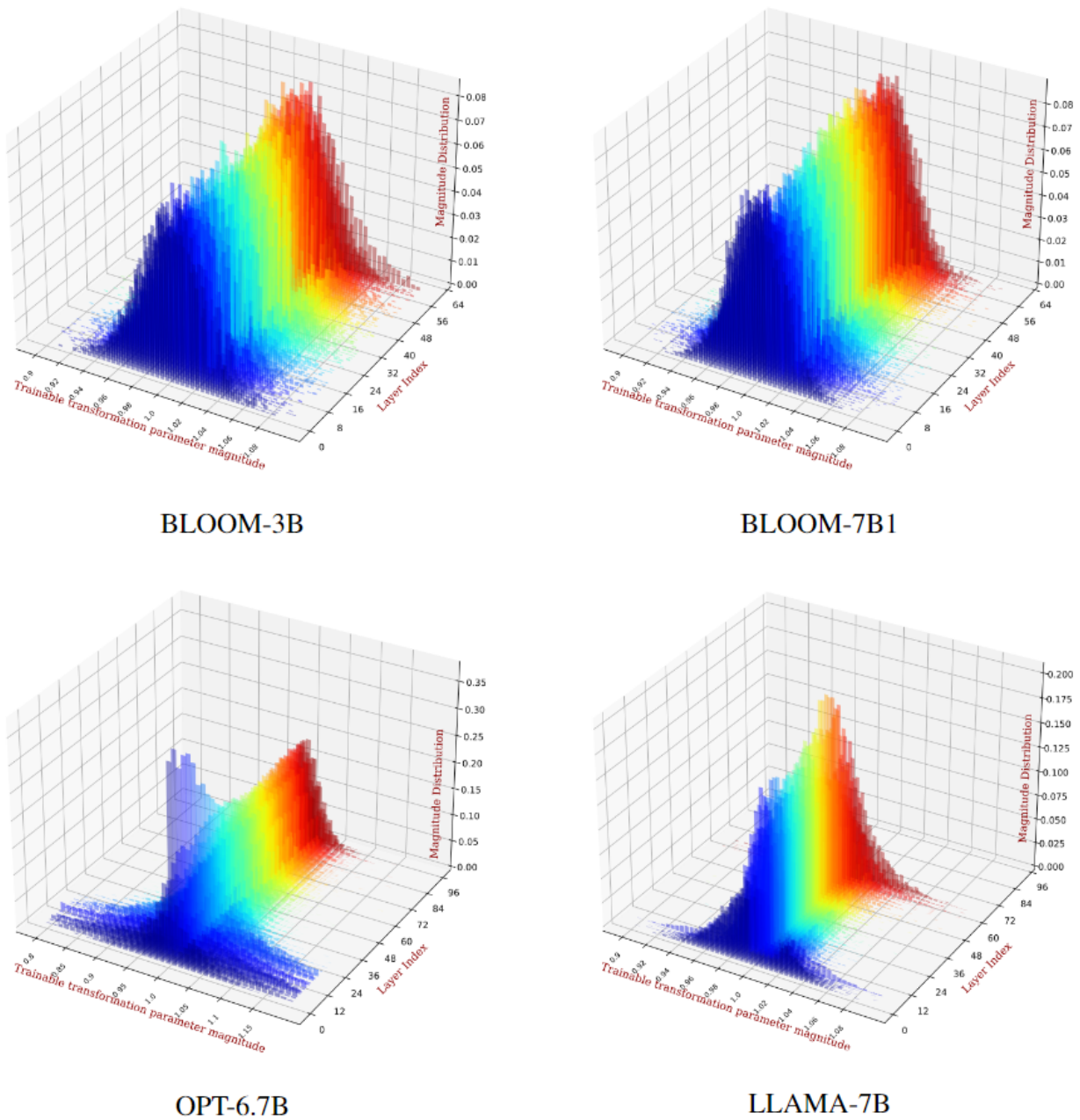



Figure 2: The magnitude distributions of scales in TEQ for **BLOOM-3B**, **BLOOM-7.1B**, **OPT-6.7B**, **LLAMA-7B**. The quantization configurations are w3_g128, w4_g128, w4, and w4 respectively. Different colors refer to layer indices in models (blue stands for shallow layers which are close to the data layer, while red stands for deeper layers).

## 5 Conclusion

In this paper, we propose TEQ, a trainable equivalent transformation that preserves the FP32 precision of the model output while also taking advantage of low-precision quantization, and its training process is lightweight. Plus, TEQ is regarded as orthogonal support for other quantization methods to improve their performance. Our task-agnostic experiments and comparison with other methods show that TEQ or its combination with other methods can obtain comparable or better results.

### 5.1 Limitations

We find that the required memory during training is still high, though the number of training parameters remains low. Moreover, since we enforce the transformation to be equivalent and keep the architecture and FP32 output unchanged, our results in some scenarios are inferior to the SOTA methods, which could be fixed by combining the SOTA methods.

### 5.2 Ethics Statement

We propose TEQ for LLMs quantization. The method can be either used individually or combined with other quantization methods. Since TEQ only requires a few steps of finetuning on original models. Thus, it is safe to say that TEQ's technical details have no significant ethical implications. Our work provides an exploration of large language model quantization through simple finetuning, making their application easier. We believe increasingly more work like this will emerge, making LLMs' quantization more powerful.

## References


Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *arXiv preprint arXiv:1607.06450*.

Haoli Bai, Wei Zhang, Lu Hou, Lifeng Shang, Jing Jin, Xin Jiang, Qun Liu, Michael Lyu, and Irwin King. 2020. Binarybert: Pushing the limit of bert quantization. *arXiv preprint arXiv:2012.15701*.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*.

Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Llm. int8 (): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*.

Steven K Esser, Jeffrey L McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S Modha. 2019. Learned step size quantization. *arXiv preprint arXiv:1902.08153*.

Elias Frantar and Dan Alistarh. 2022. Optimal brain compression: A framework for accurate post-training quantization and pruning. *arXiv preprint arXiv:2208.11580*.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*.

Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2021. A framework for few-shot language model evaluation. Github. Github: Copilot.

Ruihao Gong, Xianglong Liu, Shenghu Jiang, Tianxiang Li, Peng Hu, Jiazhen Lin, Fengwei Yu, and Junjie Yan. 2019. Differentiable soft quantization: Bridging full-precision and low-bit neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4852–4861.

Babak Hassibi, David G Stork, and Gregory J Wolff. 1993. Optimal brain surgeon and general network pruning. In *IEEE international conference on neural networks*, pages 293–299. IEEE.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778.

Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*.

Itay Hubara, Yury Nahshan, Yair Hanani, Ron Banner, and Daniel Soudry. 2021. Accurate post training quantization with small calibration sets. In *International Conference on Machine Learning*, pages 4466–4475. PMLR.

Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr.

Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

Yann LeCun, John Denker, and Sara Solla. 1989. Optimal brain damage. *Advances in neural information processing systems*, 2.

Zhengyi Li, Cong Guo, Zhanda Zhu, Yangjie Zhou, Yuxian Qiu, Xiaotian Gao, Jingwen Leng, and Minyi Guo. 2022. Efficient activation quantization via adaptive rounding border for post-training quantization. *arXiv preprint arXiv:2208.11945*.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. 2023. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv*.

Christos Louizos, Matthias Reisser, Tijmen Blankevoort, Efstratios Gavves, and Max Welling. 2018. Relaxed quantization for discretized neural networks. *arXiv preprint arXiv:1810.01875*.

Mitch Marcus, Grace Kim, Mary Ann Marcinkiewicz, Robert MacIntyre, Ann Bies, Mark Ferguson, Karen Katz, and Britta Schasberger. 1994. The penn treebank: Annotating predicate argument structure. In *Human Language Technology: Proceedings of a*

*Workshop held at Plainsboro, New Jersey, March 8-11, 1994*.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*.

Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. 2020. Up or down? adaptive rounding for post-training quantization. In *International Conference on Machine Learning*, pages 7197–7206. PMLR.

OpenAI. Openai: Chatgpt.

Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. 2016. The lambada dataset: Word prediction requiring a broad discourse context. *arXiv preprint arXiv:1606.06031*.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106.

Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. 2022. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*.

Mingzhu Shen, Feng Liang, Ruihao Gong, Yuhang Li, Chuming Li, Chen Lin, Fengwei Yu, Junjie Yan, and Wanli Ouyang. 2021. Once quantization-aware training: High performance extremely low-bit architecture search. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5340–5349.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Xiuying Wei, Ruihao Gong, Yuhang Li, Xianglong Liu, and Fengwei Yu. 2022. Qdrop: randomly dropping quantization for extremely low-bit post-training quantization. *arXiv preprint arXiv:2203.05740*.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Julien Demouth, and Song Han. 2022. Smoothquant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*.

Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. 2022. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *Advances in Neural Information Processing Systems*, 35:27168–27183.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*.

Wei Zhang, Lu Hou, Yichun Yin, Lifeng Shang, Xiao Chen, Xin Jiang, and Qun Liu. 2020. Ternarybert: Distillation-aware ultra-low bit bert. *arXiv preprint arXiv:2009.12812*.

Bohan Zhuang, Mingkui Tan, Jing Liu, Lingqiao Liu, Ian Reid, and Chunhua Shen. 2021. Effective training of convolutional neural networks with low-bitwidth weights and activations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):6140–6152.

# A Appendix

## A.1 Additional comparison with AWQ

Although both AWQ and TEQ use a small calibration set from Pile, TEQ's evaluation methodology closely follows that of GPTQ and only shares a few common tasks with AWQ. It is important to acknowledge that this comparison inherently lacks rigor due to our reliance on referencing AWQ's data alone. Consequently, this approach introduces the potential unfairness in the evaluation process, primarily stemming from the utilization of different calibration datasets.

Table 6 presents the LLaMA-7B's results of our common tasks alongside AWQ in table below and all the results of AWQ are from their paper.

## A.2 Additional comparison with GPTQ act-order

We show the results in Table 7. TEQ still outperforms GPTQ in most cases.

| LLaMA-7B | | AWQ | | | Ours | | |
|---|---|---|---|---|---|---|---|
| nbits | Method | PIQA | Hella. | Wino. | PIQA | Hella. | Wino. |
| 16 | FP16 | 78.35 | 56.44 | 67.09 | 78.35 | 56.42 | 66.85 |
| W3G128 | RTN | 75.84 | 53.10 | 63.22 | 75.68 | 53.18 | **63.06** |
| | GPTQ | 70.89 | 46.77 | 60.93 | 72.58 | 47.10 | 59.91 |
| | Proposed | **76.66** | **53.63** | **66.14** | **76.01** | **53.30** | **63.06** |
| W4G128 | RTN | 77.86 | **55.81** | 65.59 | 77.58 | **55.91** | 65.59 |
| | GPTQ | 77.20 | 53.98 | 65.67 | 77.58 | 55.83 | **66.54** |
| | Proposed | **78.07** | 55.76 | **65.82** | **78.02** | 55.76 | **66.54** |

Table 6: Reported results of AWQ and Ours

| nbits / gs | Methods | lm-eval (↑) | wikitext2 ppl (↓) |
|---|---|---|---|
| 4 / -1 | GPTQ-AO | 0.6713 | 6.06 |
| | Ours | **0.6771** | 6.30 |
| | Ours+GPTQ-AO | 0.6736 | **6.03** |
| 4 / 128 | GPTQ-AO | 0.6809 | 5.82 |
| | Ours | **0.6813** | 5.97 |
| | Ours+GPTQ-AO | 0.6811 | 5.82 |
| 3 / 128 | GPTQ-AO | 0.6042 | 8.29 |
| | Ours | 0.6521 | 6.89 |
| | Ours+GPTQ-AO | **0.6647** | **6.61** |

Table 7: Comparing results of Llama-7B for GPTQ with act-order. AO denotes act-order. TEQ still outperforms GPTQ in most cases.

### A.3 Special hyperparameters and settings

Usually, we adopt the same hyperparameters mentioned in section 3.2. So we only list all the particular settings in Table 8.

| lr | initialization | Models |
|---|---|---|
| default | $1.0/sqrt(w_{cin})$ | opt13b_w4; bloom3b_w4; bloom7b_w4; opt6.7b_w4_g128; opt13b_w3_g128 |
| 4e-4 | default | llama7b_w4; |
| 2e-4 | default | llama13b_w4_g128; |

Table 8: Special hyperparameters and settings. g denotes group size

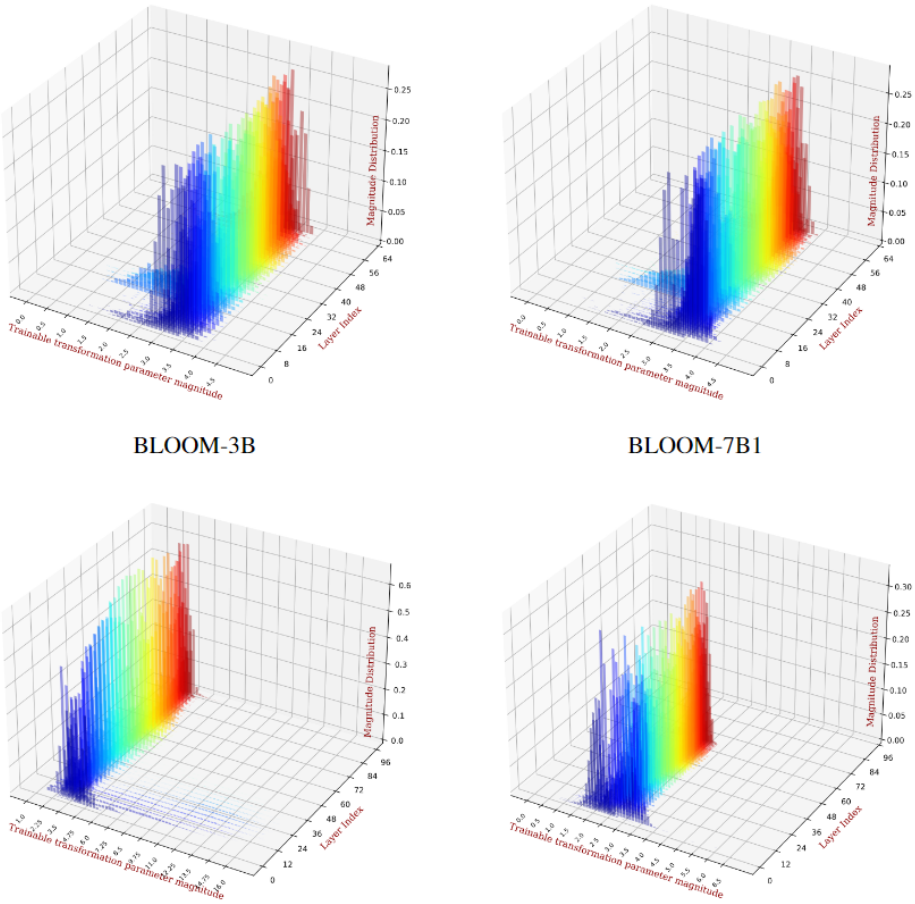
### A.4 More visualization results for TEQ's trained parameters.

Figure 3 shows the magnitude distribution of scales initialized with $1.0/sqrt(w_{cin})$. Since the initial value is related to channel-wise maximum values, it's more challenging to analyze. However, some outliers could be still observed.

### A.5 Counts of trainable parameters introduced by TEQ

We provide more details about counts of trainable parameters introduced by TEQ in Table 9. table presented below offers details regarding the applicable layers of TEQ in several models. We handle linear layers that possess transformation scales that can be assimilated by their preceding layers, such as Layer Normalization, among others.

As an illustration, within a single transformer block of OPT-6.7B, the QKV layers have the same preceding layers and therefore utilize the same set of trainable parameters. Based on the statistics, we have observed that TEQ's training only requires a minimal number of parameters (around the order from 1e-5 to 1e-4), thereby making our approach light-weighted enough.




BLOOM-3B BLOOM-7B1

OPT-6.7B LLAMA-7B

Figure 3: TEQ's trained transformation parameters' magnitude distributions, using maximum's square root value for initialization. From top to down are BLOOM-3B, BLOOM-7.1B, OPT-6.7B and LLAMA-7B respectively.

| Models | Blocks | TEQ Applicable Linear Layers | Total Linear Layers | TEQ Parameter Groups | TEQ Parameters Counts | Total Parameters Counts | Ratio TEQ params and Total Params |
|---|---|---|---|---|---|---|---|
| Bloom 3B | 30 | 60 | 121 | 60 | 153600 | 3644810240 | 0.00421% |
| Bloom 7B1 | 30 | 60 | 121 | 60 | 245760 | 8096620544 | 0.00304% |
| OPT 6.7B | 32 | 160 | 193 | 72 | 786432 | 6864388096 | 0.01146% |
| OPT 13B | 40 | 200 | 241 | 96 | 1228800 | 13110865920 | 0.00937% |
| Llama 7B | 32 | 160 | 225 | 64 | 262144 | 6738415616 | 0.00389% |
| Llama 13B | 40 | 200 | 281 | 80 | 409600 | 13015864320 | 0.00315% |

Table 9: Analysis of TEQ Parameters. TEQ only require a minimal ratio of original models' parameters (around the order from 1e-5 to 1e-4).